

Graph Creation from Relations

Rajiv Sambasivan

July 27, 2025

1 Introduction

Though we may not be aware of it, the data we work with for data analytics or machine learning use cases is often relational in nature. This means that the data we perform analysis or learning on is often a set of domain entities interacting with each other. Typically, a timestamp for the interaction is available. For example, in a retail dataset, we might have customers, products, and transactions, where each transaction links a customer to a product. Constructing a graph from such relational data can help us better understand the relationships and interactions between these entities. Operations Research (Network Models), Machine Learning (Graph Signal Processing, Graphical Models), and Data Science (Graph Analytics) all offer several canonical methods for working with graphs. Creating a graph given a set of relations puts us in a position to explore these methods and apply them to our data. The theory and practice of solving these canonical problems is well established, and if we can construct a graph from our data, we can apply these methods to gain insights, understand the characteristics of the data, and even build predictive models. If you are working with relational data, then there is a good chance that a graph perspective can benefit use case.

There is more to applying the above ideas than simply creating a graph from the data. In the context of machine learning, when working with large datasets, there is typically *heterogeneity* in the relationships between entities. In a supervised learning setting, this may manifest in how predictors are related to each other, and to the target variable. In an unsupervised learning setting, this may manifest in how the data points are related to each other. In either case, the relationships between entities can be complex and multi-faceted.

Typically, this heterogeneity can be characterized by using small number of graphs to represent our data. Each graph has the same predictor response relationship and the same relationship between predictors. In other words, rather than using a single graph to represent the entire dataset, we often have to use multiple graphs that are each homogeneous with reference to the analysis or learning task.

2 Graph Relational Learning, not a new idea

The idea of creating a graph from relations is not new. [1] is an entire textbook on the topic of graph relational learning. Applying neural networks to graphs is also not a new idea. [4] is a paper on the topic of applying convolutional neural networks to relational data using a graph perspective. The principal challenges to applying these ideas, in my experience, are

- Identifying this pattern as a standard idea to model problems where the IID assumption does not hold. There are other approaches such as hierarchical modeling that can be used to model such problems, but given the availability of graph machine learning tools, it is often easier to use a graph perspective to model such problems.
- Identifying standard ways to represent the input data as relations between entities. For example, do we want to reduce the data to a single n-ary relation, or do we want to use multiple relations to represent the data?
- Identifying the heterogeneity in the relationships between entities. What constitutes heterogeneity in the relationships between entities is application and task dependent. The methods we use to identify heterogeneity in the relationships between entities is different in a balanced classification task (target variable distribution is balanced) and an unbalanced classification task (target variable distribution is unbalanced). The methods we use for unsupervised learning problems are different from the methods we use for supervised learning problems.

Once we have created a graph from the relations and have identified the heterogeneity, then there are canonical problem representations, such as node classification/regression or link prediction that we can use to model the problem. The methods for solving these problems are well established, and we can apply them to our data. Standardization of the data representation and the heterogeneity identification is key to applying these methods effectively. Heterogeneity identification requires a combination of domain knowledge and data analysis techniques. The next section provides examples of how to create a graph from relations, and how to identify heterogeneity in the relationships between entities. The first example applies unsupervised learning to develop a descriptive model of customer purchasing behavior in Sao Paulo state, Brazil. The second example applies supervised learning to develop a predictive model of loan performance for loans guaranteed by the Small Business Administration (SBA) in the US. Both examples illustrate how to create a graph from relations and how to identify heterogeneity in the relationships between entities. As will be seen, what constitutes heterogeneity in the relationships between entities is different for the two examples.

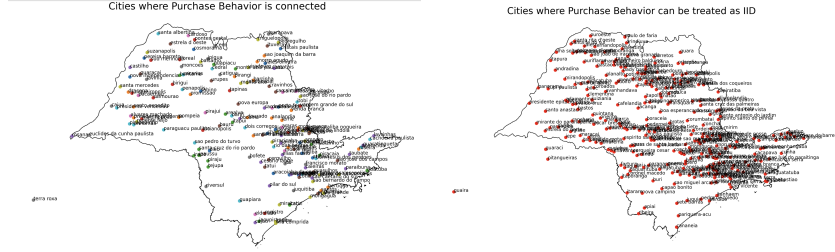
3 Examples of Graph Creation from Relations

I will provide two examples of how to create a graph from relations. In both these examples, I will point out the two points made in the previous section: (1) The data is a set of relations between entities. (2) The data can be viewed as a set of graphs.

The first example is the [Olist Case Study](#). In this example the relations are between customers, products, and purchases(invoices). The description shows two summary perspectives of the data: (1) Temporal perspective, where the data is summarized by week of the year for the year 2017. (2) Spatial perspective, where the data is summarized by state in Brazil. This is an example of heterogeneity arising from different tasks (perspectives) on the same data. The summary shows that the data is relational in nature, and that it can be viewed as a set of graphs. The temporal perspective can be viewed as a graph where each week is a node, and the edges represent the relationships between weeks. The spatial perspective can be viewed as a graph where each state is a node, and the edges represent the relationships between cities in the state. The combination of these two perspectives can be viewed as a graph where a graph is produced for each state and each week, the spatio-temporal graph, can also be produced, but not done in this example. Graph signal processing methods can be applied to analyze, decompose, and summarize a signal such as the revenue on the nodes of the graph. The graph can also be used to build predictive models, such as forecasting revenue for a given week in a given state. The latter (predictive modeling) is not a descriptive task, but it builds on the structure developed in the descriptive task for predictive modeling. Both of these tasks are not done in the example. I will probably do a follow up post on this example to show how to apply graph signal processing methods to the data.

For the spatial perspective, the data is summarized by city in Sao Paulo state. The cosine similarity in sales is used to create a graph. Cities with similar sales are connected by an edge. The threshold used here is a simple threshold, but a range of techniques can be used to create the graph. Passing the similarity through an affinity kernel is one such technique. Basically, the similarities associated each city are passed through a kernel function to create a graph. Another approach is to use fixed neighborhood size, for example, five nearest neighbors. As a result of applying a suitable technique to select the edges associated with each city, we create a graph that captures the relationships between cities in Sao Paulo state. The connected components of this graph provides the heterogeneity in the relationships between cities. Some cities are not connected to any other city, while others are connected to several other cities. The ones not connected to any city are singleton connected components, while the ones connected to several other cities are larger connected components. The singleton connected components as a group can be analyzed using methods suitable for analyzing Independent and Identically Distributed (IID) data, while the larger connected components can be analyzed using methods suitable for analyzing graphs. *As a consequence, we have two different analysis approaches to understand customer purchasing behavior in Sao Paulo state.* One for the

IID data and one for the connected graph data. The figures below are from that case study



(a) Schematic of Geographic Region for Graph Analysis (b) Schematic of Geographic Region for IID Analysis

The ideas used to identify heterogeneity in the relationships between entities is application and data dependent. There is no one size fits all approach to identifying heterogeneity in the relationships between entities. The key is to use domain knowledge and data analysis techniques to define what constitutes heterogeneity. For unsupervised learning problem, we can leverage covariance or similarity to identify heterogeneity. This is the case for the Olist example. For supervised learning problems, some parts of the data are easier to predict than others. For example, we can have regions of the data that are associated with a single label. These regions have a simple classification rule, see the 1-R classification rule [2]. Other regions may require more sophisticated classifiers. These other regions can be profiled by a By adjusting the height of the tree and checking that results generalize, the dataset can be partitioned into regions (the leaves of the tree) that are each homogeneous from the standpoint of the learning task. For more details, please refer to [3].

The second example comes from the [SBA website](#). The data is the performance of a certain type of loan guaranteed by the Small Business Administration (SBA) in the US. For more information on these type of loans, see [this page](#). The data is a set of relations between loans, borrowers, and lenders. The SBA reports the performance of these loans on a monthly basis. An important perspective of the data is the performance of loans by month. The loans span different geographic regions and a range of industries. The SBA guarantees the loans, so it may be important to understand the kind of regions and industries that are associated with good and bad performance each month. There is a notebook that covers how the data associated the SBA loans is cleaned and prepared for this specific analysis task.

For this specific analysis task, there is heterogeneity in the predictor response relationship for different parts of the data. Some Borrower regions have no defaults for example. If a borrower is from these regions, then the SBA knows that the loan will not default. Other regions are a mixed bag and need a more sophisticated model to predict the performance of the loan. A notebook that covers the details of preparing the data for this analysis task is available at [here](#).

[illegible]

For this example, I have a sketch of an idea that I hope to abstract away into a generic tool for creating graphs from relations. This is discussed in the next section.

The input we get for tabular operational data is typically one of the following:

- We can then specify the entities and the relationships between the entities in the relation. This can be specified in a configuration file. The configuration file can be a simple yaml file that specifies the entities and the relationships

between the entities. For example, an excerpt from configuration file the SBA example is shown below:

```
relations:
  borrower_to_bank:
    from: "Borrower"
    from_key: "BorrName"
    to_key: "BankFDICNumber"
    to: "Bank"
    description: "A borrower can have multiple loans"

  bank_to_loan:
    type: "many_to_one"
    from: "Bank"
    from_key: "BankFDICNumber"
    to_key: "LoanID"
    to: "Loan"
    description: "A borrower can be associated with multiple lenders across
different states"
```

Figure 3: Heterogeneity, Good Borrower Loans (limited to California)

The graph derived from this definition is shown below.

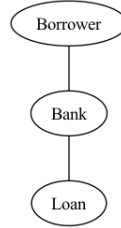


Figure 4: Heterogeneity, Good Borrower Loans (limited to California)

So a possible approach to standardizing the process of creating a graph from relations is the following:

- Read the input data and construct a single n-ary relation $\mathbf{R}$.
- Perform data cleaning and preprocessing on the relation $\mathbf{R}$.
- Do the data analysis to identify the heterogeneity in the data.
- Create a configuration file that specifies the entities and the relationships between the entities in the relation $\mathbf{R}$ for each subset of the data that is homogeneous with respect to the analysis or learning task.
- Use the configuration file to create a graph from the relation $\mathbf{R}$.
- Perform the analysis or learning task on the graph.

5 Conclusion

This is an initial draft of a process to analyze and learn from relational data that is heterogeneous in nature. The heterogeneity can be identified by using domain knowledge and data analysis techniques. The process of creating a graph from relations is a key step in this process. The examples provided illustrate how to create a graph from relations and how to identify heterogeneity in the relationships between entities. The next steps are to refine the process and to develop a generic tool for creating graphs from relations. Adding more rigor and examples to the process of identifying heterogeneity in different machine learning tasks is work in progress.

References

- [1] Lise Getoor and Ben Taskar. *Introduction to statistical relational learning*. MIT press, 2007.
- [2] Robert C Holte. Very simple classification rules perform well on most commonly used datasets. *Machine learning*, 11(1):63–90, 1993.
- [3] Rajiv Sambasivan and Sourish Das. Classification and regression using augmented trees. *Int. J. Data Sci. Anal.*, 7(4):259–276, 2019.
- [4] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *European semantic web conference*, pages 593–607. Springer, 2018.